Ivy is Hard to Implement

Minh Vo
m9vo@ucsd.edu
UC San Diego
La Jolla, California, USA

Eric Van Grinsven
evangrin@ucsd.edu
UC San Diego
La Jolla, California, USA

ABSTRACT

We find out that the Ivy distributed shared memory system is not quite as simple to implement as the algorithms on the paper would seem to suggest

1 INTRODUCTION

We implement IVY in C and run each process on instances of AWS's t2.micro. We took the recommended approach of using `mmap` to set aside a region of memory of a fixed size, and then `mprotect` these regions, using our own fault handler to enact the calls that the IVY algorithms describe for the distributed shared memory. We currently have written the centralized manager algorithm for IVY, and are in the process of creating more tests for our system. There are a few small bugs we are currently dealing with that will be detailed in a later section. Section 2 talks about the background of IVY. Section 3 talks about the specific design of our IVY user-level distributed shared memory system. Section 4 talks about implementation details.

2 BACKGROUND

Ivy is a distributed shared memory system with sequential consistency. In the IVY paper, much details are discussed about its consistency model (sequential) and cache coherence. It also details 3 implementations – centralized manager, distributed manager, and distributed manager and copyset. We will forego writing more for the background for the checkpoint, since we don't think it is necessary to understand our progress, but we will detail more about what the paper talks about as we make more progress on implementing the different versions of IVY and benchmarking them.

3 DESIGN

In our current implementation of IVY, we are using the centralized monitor-like algorithm, as described in Figure 2 in [1]. A processor in our implementation can be a process running on a machine, so processes can run of different machines, or one machine can have multiple processors as well. Each processor is identified by an number starting from 0, and an IP address and port. Currently, we designate processor 0 as the manager. We make an assumption that processors using IVY all know each other beforehand, so there is a configuration file with all IP addresses and ports that is read on initialization.

Our user-level implementation of IVY pre-allocates all pages in the distributed shared memory region, and sets access rights to each page in that region. The shared memory region is hard-coded to begin at a specific logical address, which we will call the base address of the distributed shared memory region. Any access to a page without the correct access rights will cause a segmentation fault, which we catch and service using a custom fault handler. Each processor will also listen to requests on their assigned IP and port in order to carry out requests by some requesting node after a fault occurs.

Currently, we designate that each processor is initially responsible for a set number of pages (currently set to 1024). Each processor is initially responsible for successive sets of pages, so processor 0 (the manager) starts out owning pages 0-1023, processor 1 starts out owning pages 1024-2047, and so on. Ownership changes over the course of execution as processors request to write to pages. Because of this setup, we currently assume that memory accesses are safe. This means that if processor i isn't running at the start, then no other processors would attempt to use pages initially owned by i (as this would result in a connection error). Similarly, if processor i shuts down, then it owns no pages. In addition, we also assume that the manager starts before all other processors start. These assumptions are made because the IVY paper only describes a solution to the cache coherence problem, not fault-tolerance.

4 IMPLEMENTATION

We implement the user-level IVY system using C on AWS's t2.micro instances. Since we know the size of the shared memory region beforehand, we use `mmap` to set aside a region of size $num_procs * num_pages_per_proc * page_size$ bytes. As mentioned, $num_pages_per_proc = 1024$, and $page_size = 4096$ bytes, the same as the page size on t2.micro. Each page is protected using `mprotect`, which we use to set a page to `PROT_READ` (read access), `PROT_WRITE` (read access), or `PROT_NONE` (none). The fault handler we use can detect the address of the fault, which we can use along with the base address (currently set to address 2^{19}) to find the faulted page number. The fault handler can also detect if the fault is caused by a read or write, so we can call the correct fault handler from IVY.

There are some global variables in our implementation of IVY, due to the restrictive nature of the custom fault handler function. Since the fault handler, to our understanding, does not allow us to pass in extra data, we made variables that are used in the fault handler global, so that our fault handlers can access them. These variables currently include the page table, the manager's info table, the process table (storing IP addresses and ports), the number of processors, the ID of the running processor, and the pointer to the base of the shared memory region.

The listening server runs as a separate thread in the current processor. The thread starts up after the shared memory region is setup, and listens on the processor's IP address and port for requests. Requests are received in the form of operation, page number, requesting node, each separated by a null terminator. As requests are received, it is serviced by the read server, write server, and invalidate server, as detailed in the IVY paper.

One deviation we made from the algorithm that IVY proposes is in the sequence of messages that get sent on a page request. In the algorithm described by IVY, the chain of messages goes as follows: (assuming node i requests the page, and node j owns the page).

(1) i requests the page from the manager
(2) manager passes the message to j
(3) j passes the page to i
(4) i updates manager that it has received the page

This gives the sequence of $i \rightarrow m \rightarrow j \rightarrow i \rightarrow m$.

Our current implementation does the following:

(1) i requests the page from the manager
(2) manager passes the message to j
(3) j passes the page to m
(4) m passes the page to i, ending the sequence

This gives the sequence of $i \rightarrow m \rightarrow j \rightarrow m \rightarrow i$. This is because we do not know of a way to do a 3-way message chain in C. When the requesting node opens a connection with the manager, it can use the same connection to receive messages from the manager. On the other hand, in the IVY algorithm, the requesting node opens a connection with the manager, but then has to listen from the page owner. We can have the owner send a page through the requesting node's IP address, but then we would have to find a way for the listening thread to communicate with the fault handler that the page has been received, which we do not know how to do either.

There should be no change in correctness. In addition, since the manager is the processor that passes the page to the requesting node, the requesting node wouldn't have to send an acknowledgement of receipt. The message count is the same, however we note that the amount of traffic sent is higher, since we now forward the page twice instead of just once. In addition, it does not deal with the bottleneck issue caused by having a centralized manager, and may actually worsen this issue since now the manager is a part of one extra message in the communication sequence, in addition to the traffic amount.

When we move to the distributed manager algorithm, we think this will get worse, because then, the page is requested recursively through the chain of probOwners. This means that the requesting node will request the probOwner it knows, and after a chain of requests, the owner would then have to forward the page back through the chain. We have an idea of how to reduce the actual message load, where instead of recursively forwarding the request, we do it iteratively instead. The idea comes from the diagram for recursive vs. iterative DNS resolution that we saw in a networked service class. Figure 1 demonstrates this idea. In the iterative version, we only forward the page once, compared to 4 times in the recursive version. We will have to check if this works, but if it does, it would reduce the amount of data sent between processors, which will improve the system performance. Though if we can implement something like the pseudocode in the paper, that would also reduce the amount of messages.

5 CURRENT PROGRESS, CHALLENGES AND PLANS

Currently, we have a (generally) working implementation of IVY using a centralized manager, which we test by having a set of processors initializing IVY, then hangs around to service requests. Specifically our testing process was as follows:

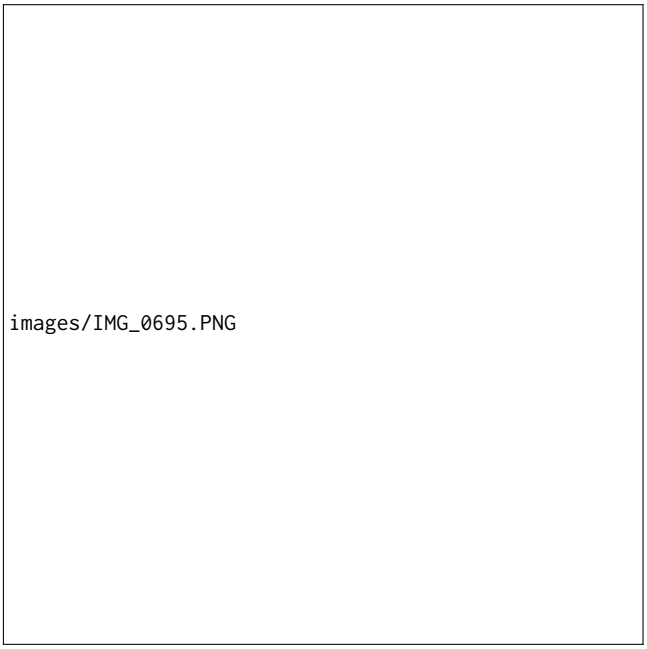


Figure 1: Recursive (left) and iterative (right) page requests

(1) Processors 0 through 4 start up, initializes IVY, then hangs
(2) Processor 5 starts up, initializes IVY, then attempts to read, then write, to a page owned by processors 0 through 4. On reads, we should see debug statements saying that processor 5 got added to the copyset, but ownership remains the same. On writes, we should see ownership transfers from processor 0 through 4 to processor 5. In addition, we should also see the initial read values be garbage (normally just the null terminator), but any read after the write should reflect the written value (currently we write the letter 'a').
(3) After this is verified, processor 6 starts up, initializes IVY, then attempts to do the same thing as processor 5. We verify the same things, but now, we check if values previously written by processor 5 is read by processor 6. We also check that all ownership transfers come from processor 5, instead of 0 through 4.
(4) We repeat the previous step a few times for different processors, and observe read values, and ownership transfers.

Currently, sometimes our code results in a connection refused error. We believe that we found a fix, since we realized that we set up the listening socket in the thread function, which might not have been scheduled to run when other processors start sending requests, which causes a connection failure. We fixed this by starting the socket first, then pass it into the thread function. We believe this took care of the issue, but should the problem comes up again, we will attempt to fix it.

For the remainder of the quarter, we hope to split responsibility between writing benchmarks for the system, and implementing the distributed version of IVY (starting with distributed manager, then

distributed copyset if time permits). By the start of week 10, we plan to have a working implementation of the distributed version of IVY (preferably with the distributed copy-set). We also plan to have more extensive tests by this time in order to ensure correctness of our implementation. Throughout week 10, we plan to benchmark our implementations of the distributed and centralized managers. We believe that as we put more work into the benchmarking and distributed manager algorithm for IVY, we will continue to have more ideas , such as minor changes to implementation of the algorithms (such as what we described earlier for our centralized manager implementation) that will make our project interesting and give us new insights. One current idea is benchmarking write contention, since our page size is larger than the recommended size, which might cause a lot of false sharing. Another we're thinking about is seeing the difference between growing memory as processors are added, or fixed memory but each processor is responsible for a smaller region, as the number of processors grow.

Lastly, since the IVY system mostly deals with ensuring consistency, we would like to explore more regarding fault tolerance, if there is time. We would like to see if it is possible to implement a version of IVY where if certain nodes don't start up, then other nodes can take over responsibility initially (in a Chord-like fashion similar to lab 3). However, we realize this might be a lot of work (as the labs did), and doing it in C might be challenging.

REFERENCES

[1] Kai Li and Paul Hudak. 1989. Memory Coherence in Shared Virtual Memory Systems. *ACM Trans. Comput. Syst.* 7, 4 (nov 1989), 321–359. https://doi.org/10.1145/75104.75105